National College of Ireland

Project Submission Sheet

Student Name:	Wasiullah Khan		
Student ID:	X24217760		
Programme:	MSc Cloud Computing	**Year:**	2025-2026
Module:	Fog and Edge Computing		
Lecturer:	Estrada		
Submission Due Date:	17th April 2026		
Project Title:	Fog-Assisted Smart Waste Monitoring		
Word Count:	4029		

Academic Integrity Certificate Verification Code: **SKizPX9EfK**

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.
ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement – Starting **Monday, 19th January 2026**, **all students are required to successfully complete the Epigeum Academic Integrity Course**, which is available on **Moodle**. This is a **self-paced online course** consisting of **4 modules** and will take approximately **2 hours** to complete. Once you have successfully completed the course, you will receive a **Certificate & Verification Code**. This code **must be included on the Project Cover Sheet** when submitting your work. For full details and guidance, please visit: 🔗 ttps://libguides.ncirl.ie/academicintegrity/epigeumforstudents

Signature:	Wasiullah Khan
Date:	13th April 2026

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.
3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**

5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplments in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

[Insert Module Name]

[Insert Title of your assignment]

Your Name/Student Number	Course	Date

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click here.

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used**.

[Insert Tool Name]	
[Insert Description of use]	
[Insert Sample prompt]	[Insert Sample response]

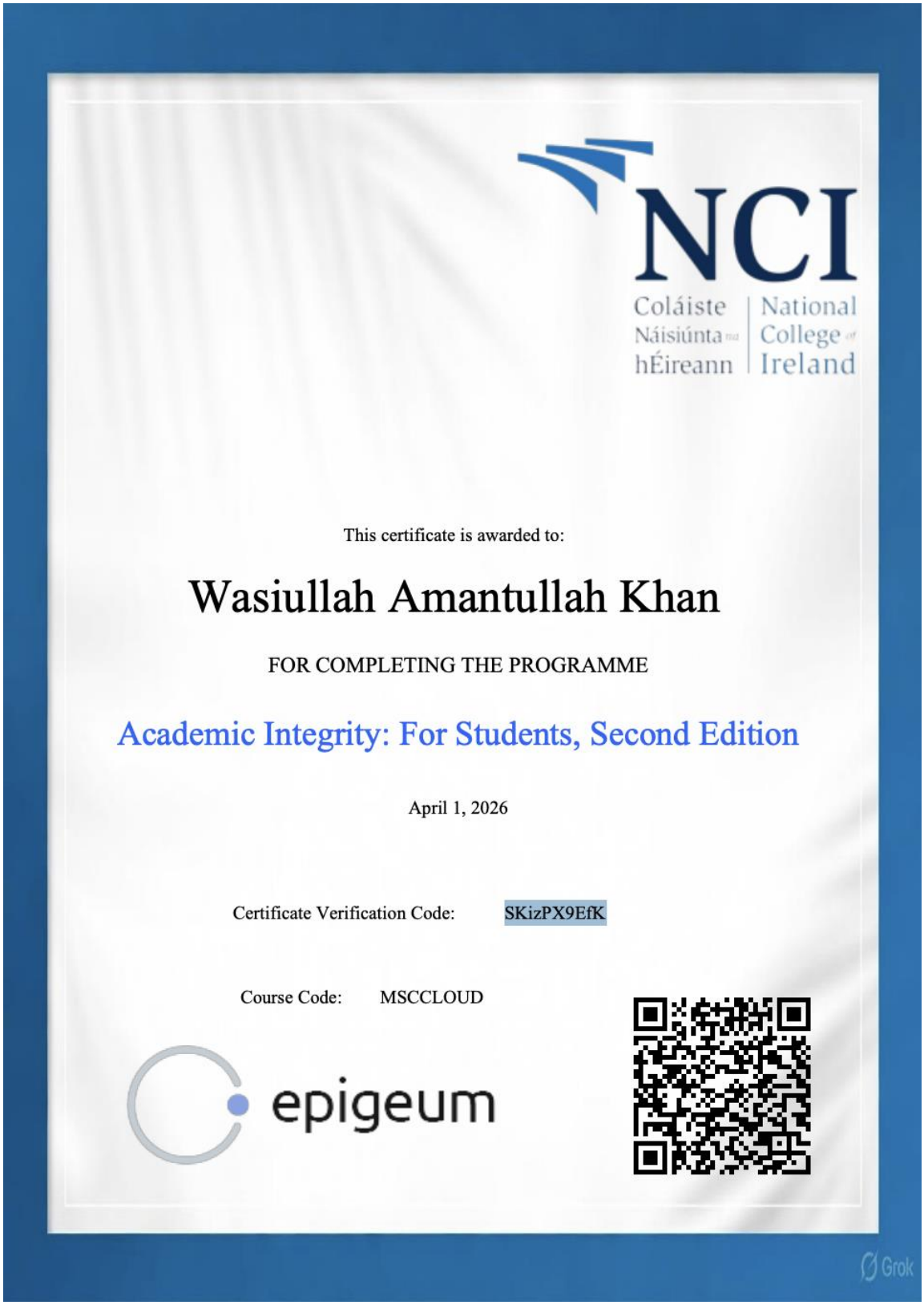
Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

Additional Evidence:

[Place evidence here]

Academic Integrity Acknowledgement Supplement



NCI
Coláiste Náisiúnta na hÉireann | National College of Ireland

This certificate is awarded to:

Wasiullah Amantullah Khan

FOR COMPLETING THE PROGRAMME

Academic Integrity: For Students, Second Edition

April 1, 2026

Certificate Verification Code: SKizPX9EfK

Course Code: MSCCLOUD

epigeum

Grok

Fog-Assisted Smart Waste Monitoring: A Serverless Pipeline from Simulated Edge to AWS

Wasiullah Khan
Fog and Edge Computing
x24217760
National College of Ireland

Deployed App: https://main.d12jcq31pcipw0.amplifyapp.com/

Base API URL: https://zzxm3e4tq1.execute-api.us-east-1.amazonaws.com

GitHub URL: https://github.com/Wasiullah26/Smart-waste-bin

***Abstract*—Managing waste in smart cities depends heavily on the having real-time data about bin status and all—knowing when a bin is close to overflowing or when the heat is building up in an enclosed space can make all the difference for the operations teams. This project builds a *fog-first* pipeline where the data gets processed at the edge first, and then gets refined at the fog layer, and only then pushed up to the cloud. A Python-based programmed simulator acts as a stand-in for real hardware (sensor), generating multi-signal readings across several zones and bins—things like fill level, weight, temperature, and usage counters. The fog layer handles the smoothing via a moving average, calculates a weighted priority score, and classifies the operational state of each bin (including escalating from the `fire_risk` to `probable_fire` when conditions worsen), and packages everything into the versioned JSON before sending it off to the cloud ingest endpoint. On the AWS side, API Gateway picks up the incoming requests, an ingest Lambda validates the incoming data and then queues them via AWS SQS, and a processor Lambda writes both the event history and the current state into the DynamoDB—with a GSI (Gobal Secondary Index) to handle the zone-based queries. A React SPA (Single Page Application) hosted on the AWS Amplify pulls from a read Lambda to display the fleet dashboard on the web. The whole thing leans on the asynchronous decoupling between the API and the storage layers, pay-per-use serverless services, and AWS SAM for deployment under Learner Lab constraints. Evaluation was done through manual `curl` testing, DynamoDB and CloudWatch inspection, and UI walkthroughs rather than formal load testing. The system is repeatable and the key design decisions are documented throughout, including some hard-won lessons around credential rotation, region consistency, and frontend API configuration.**

***Index Terms*—Fog computing, Internet of Things, serverless computing, AWS Lambda, Amazon DynamoDB, edge preprocessing, smart waste, systems integration.**

I. Introduction

The Fog and Edge Computing module raises a central question: where should the computation actually happen when the devices are producing thhe data continuously? And purely cloud-based approaches have the obvious appeal, but they come with trade-offs too—latency to actuators, bandwidth costs, and dependence on stable connectivity. Although the local gateways can help, and splitting work across the layers often makes the most sense. Waste management is a useful setting for exploring these tensions. Bins are spread out geographically all around the cities, fill levels shift across days and seasons, and temperature spikes—while rare—can signal the genuine safety concerns. If every raw reading generated by the bins gets sent straight to a regional API, storage costs will keep climbing and simple questions like "is this bin trending bad?" becomes dependent on round-trip times to the cloud.

This report covers a **smart bin fleet** demonstrator built as part of the continuous assessment. **Edge** responsibilities run locally on a developer machine: that means the synthetic sensor data generation and fog preprocessing. **Cloud** responsibilities run on the AWS: HTTP management, message buffering, durable storage, and hosting for the dashboard. Physical devices like the MQTT sensors or LoRa gateways were out of scope here—but we have the simulator which is there to provide the controllable, repeatable data streams so validation and scoring logic can be tested without needing actual hardware or actual sensors.

A. Problem framing and questions

Three questions that shaped the design process. **Q1 (Placement):** What work needs to be happen before the cloud receives data, to keep the downstream costs and noise manageable? **Q2 (Coupling):** How should the synchronous HTTP path relate to asynchronous persistence so that the demos remain stable even under the bursts of traffic or intermittent connectivity? **Q3 (Verifiability):** How can the fog and the cloud agree on the payload structure so that breakages are caught early, in CI, rather than surfacing during a live demo?

B. Contributions

The contributions here are the engineering and documentation rather than algorithmic novelty: (i) a versioned ingest contract shared between the fog emission, backend validation and the automated tests; (ii) clear priority and thermal thresholds with a documented escalation path; (iii) a SAM-defined multi-Lambda stack with SQS sitting in between ingest and persistence for queuing; (iv) a TypeScript React UI with the routes aligned to the read APIs; (v) GitHub Actions plus the AWS Amplify build automation. Section II sets out the

requirements. Section III surveys relevant background work. Section IV covers the architecture. Section V goes deeper on each subsystem. Section VI maps implementation decisions to the repository. Section VII covers testing. Section VIII evaluates outcomes and limitations. Section IX wraps up with conclusions and future directions. Appendices include payload examples, field inventories, and operational checklists.

II. Problem Statement and Requirements

A. Stakeholders and scenario

In an actual deployment when it is ready for the production with the real sensors, the main stakeholders would be municipal operations staff handling the routing, maintenance crews responding to overflow or heat alerts, and systems integrators consuming the API. For the purposes of this coursework the scenario is simpler: a single developer-operator demonstrating end-to-end behaviour to the markers—showing that the telemetry enters the cloud, and that state is visible in DynamoDB, and that the public dashboard reflects the same data you'd get from a `curl` command.

B. Functional requirements

1) **FR1 — Multi-signal generation:** Generate per-bin streams covering fill (%), mass (kg), temperature (°C), and usage counters at the configurable intervals, across multiple zones and many bins per zone.
2) **FR2 — Fog processing:** Validate those incoming readings; maintain a bounded rolling windows for the smoothing; compute a priority score and discrete status; send the `summary` or `alert` messages as appropriate; avoid the flooding of the API with repeated POSTs call when nothing has changed.
3) **FR3 — Cloud ingest:** Expose an HTTPS POST endpoint; validate incoming JSON against a strict schema; return clear, and useful errors for malformed bodies; enqueue the durable work items; complete the request without blocking on full DynamoDB fan-out.
4) **FR4 — Persistence and query:** Append the telemetry history records; maintain a latest-state projection of per bin; support the fleet listing; support fetching detail for individual bins; support for the zone-scoped queries through an index; expose a critical-attention list; return recent history for sparkline-style charts.
5) **FR5 — Dashboard:** Provide a SPA (single page applciation) with fleet, detail, and critical views; render status badges consistently with the API enums and booleans—fill stress, fire risk, and probable fire should all be clearly visible.
6) **FR6 — Deployability and repeatability:** Document all build-time environment variables; supply a SAM template and CI checks; support static hosting via Amplify through the github.

C. Non-functional requirements

NFR1 — Elasticity: Prefer managed scaling (Lambda, on-demand DynamoDB) over the fixed-size instances. **NFR2 — Cost:** Minimise the always-on charges given the low traffic of a demo environment or would be used by the admins only for all organizations. **NFR3 — Maintainability:** Keep to a single infrastructure template; share Python types and validators across components where practical. **NFR4 — Observability (lightweight):** CloudWatch logs for each Lambda as the first line of debugging. **NFR5 — Security (coursework tier):** TLS for all the transport; an optional admin key for destructive routes; the template explicitly marks read paths as open.

III. Background and Related Work

A. Fog and edge paradigms

Fog computing places intermediate resources between end devices and cloud data centres with the goal of reducing the latency and pressure on the core network [2]. Architectural surveys covers the gateways that preprocess, filter, or aggregate the data streams before sending them on to the cloud [3]. Edge computing surveys covers the similar placement trade-offs from the device side [13]. Atzori et al. put the device proliferation and the heterogeneous protocols in the broader context [1]. Research tools like the iFogSim can model the resource management across fog tiers [12], though this project takes a more direct approach—implementing the pipeline rather than simulating it at an abstract level. In the real deployment, "locally" would mean two distinct layers: the sensors which are attached to the bins themselves handling basic validation at the device level, and a fog gateway sitting somewhere on the local network—maybe a street cabinet or a municipal building—running the heavier smoothing and scoring logic before anything leaves the area. Here, both roles are collapsed onto a single laptop: the pyhton based simulator stands in for the bin sensors and the fog script stands in for the gateway. The architecture and the design decisions are the same either way, which is precisely why this kind of the simulation is reasonable and accepted substitute when physical hardware is out of scope.

B. Protocols and industrial practice

MQTT is the most commonly used publish–subscribe protocol at the edge [4], and many industrial stacks pair MQTT brokers with cloud analytics backends. This project skips the broker in favour of the HTTPS JSON directly from the fog process to the cloud API. The trade-off is that the less flexibility in terms of message patterns, but the gain is that its much easier debugging—you can replay any payload with `curl` and can easily see exactly what happens.

C. Serverless IoT integration

AWS provides API Gateway, Lambda, SQS and DynamoDB as composable building blocks for the serverless IoT backends [6]–[9]. AWS SAM encodes these compositions as infrastructure-as-code [5]. The citations in this section are mainly for documenting engineering choices rather than making any claim to the research novelty.

D. Client engineering

The dashboard uses React with Typescript for component composition and Vite for the build tooling [10], [11]. TypeScript adds the static types for API response shapes, which helps catch UI drift when API fields change or gets renamed.

IV. Architecture and Design

A. Layered reference model

Figure 1 shows the major layers of the system. **Edge source:** NDJSON output from the python based simulator. **Fog:** smoothing, scoring, and packaging. **Regional control and data plane:** API Gateway, the ingest Lambda, and SQS. **Persistence:** the processor Lambda and DynamoDB tables with indexes. **Read plane:** the read Lambda. **Presentation:** the browser SPA hosted on AWS Amplify.

B. Ingest path and coupling

The ingest Lambda does the light validation and enqueues a message—that's it. Then the processor Lambda picks up the batches asynchronously. This split is the answer to Q2: if the simulator is running in a tight loop or the laptop's Wi-Fi drops for a few seconds, the queue absorbs the work without causing API Gateway timeouts [8]. In a real deployment, the same principle applies at the different scale—dozens of fog gateways across different neighbourhoods could all be pushing readings simultaneously, and without a queue sitting in the middle, a sudden spike in traffic or a brief network hiccup on any one gateway could cascade into the dropped requests or timeout failures at the API layer. The queue decouples the rate at which the data arrives from the rate at which it gets written to the DynamoDB, meaning each layer can operate at its own pace without one blocking or overwhelming the other. The downside is eventual consistency—there's a short gap between the message being enqueued and the corresponding DynamoDB row becoming visible. That's acceptable when the dashboard is polling at intervals measured in seconds.

C. Read path

Read routes are implemented in a separate function from the ingest and processor. This limits the blast radius if read and write traffic patterns diverge. CORS settings in the SAM template are deliberately permissive for coursework purposes; a production deployment would restrict the allowed origins and would require authentication on at least some routes.

D. Data modelling rationale

Maintaining both an **event history** table and a **latest-state** table is a deliberate: event history supports the auditing and the short-window trend views, while the latest table avoids having to scan the history just to render the fleet overview screen. The **zone GSI** makes `/zones/{zone}/bins` work without full-table scans [9]. TTL on the history events could be added later once data volumes grow.

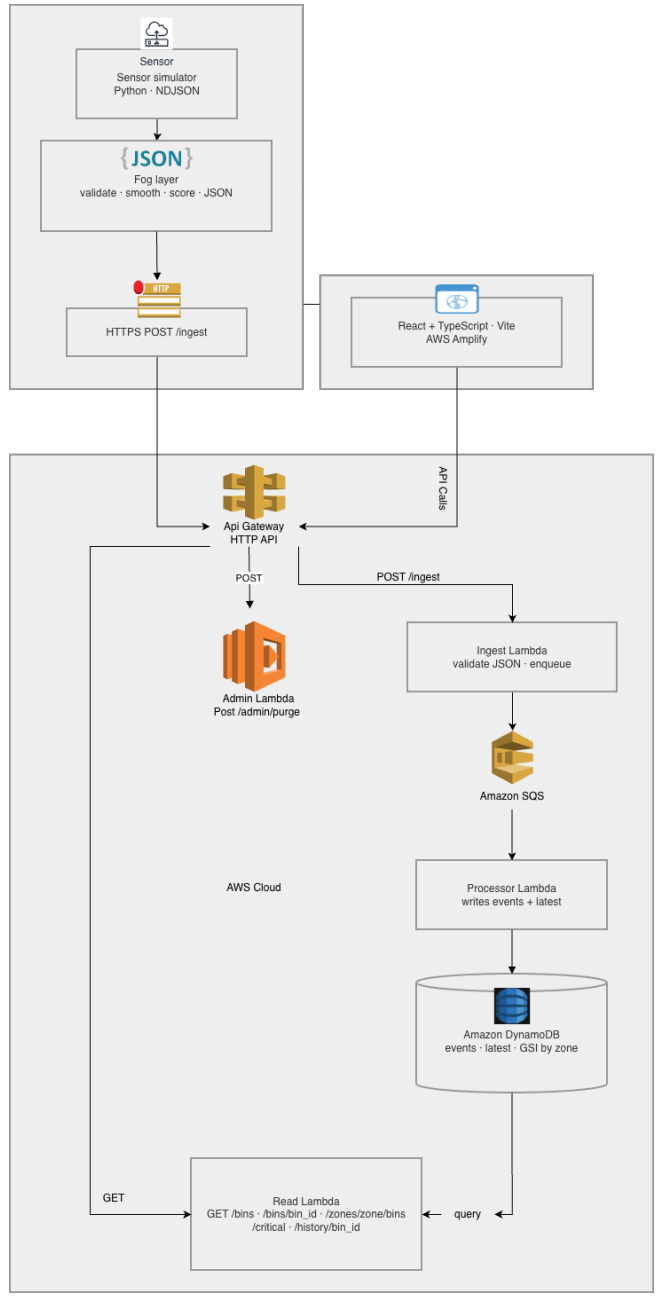


Fig. 1. System architecture: simulated edge (sensor + fog) to AWS (API Gateway, SQS, Lambdas, DynamoDB) and Amplify-hosted dashboard.

E. Fog scoring and thermal policy

Let $f \in [0, 100]$ be fill (%). Usage and temperature are normalised to $u, t \in [0, 100]$ as follows:

$$u = \min\Big(100, \max\big(0, \frac{\text{usage}}{\text{CAP}} \cdot 100\big)\Big), \tag{1}$$

$$t = \min\Big(100, \max\big(0, \frac{T - T_{\min}}{T_{\max} - T_{\min}} \cdot 100\big)\Big), \tag{2}$$

where CAP=30, $T_{\min}$=5 °C, and $T_{\max}$=55 °C. The composite priority score is then:

$$P = \text{clamp}_{[0,100]}\big(0.6\, f + 0.3\, u + 0.1\, t\big). \tag{3}$$

TABLE I
KEY CONSTANTS (SHARED/CONSTANTS.PY).

Parameter	Value
WEIGHT_FILL, WEIGHT_USAGE, WEIGHT_TEMP	0.6, 0.3, 0.1
FILL_WARNING_PCT, FILL_CRITICAL_PCT	80, 95
FIRE_TEMP_THRESHOLD_C	45
PROBABLE_FIRE_TEMP_THRESHOLD_C	60
USAGE_COUNT_CAP	30
TEMP_MIN_C, TEMP_MAX_C	5, 55
SMOOTHING_WINDOW	5

Fill alerts are triggered at 80% (warning) and 95% (critical). On the thermal side, readings at or above $45\,^{\circ}$C which triggers `fire_risk` and readings at or above $60\,^{\circ}$C triggers `probable_fire`, with both booleans propagated in the payload for the UI to display as badges.

F. Alternatives considered

MQTT broker: More realistic for actual device fleets but it comes with thee operational overhead and that wasn't justified here. **Kinesis Data Streams:** Well-suited for the analytics pipelines; SQS was sufficient for the decoupling requirement. **GraphQL:** More flexibility on the read side, but REST kept the Lambdas and SPA simpler to reason about. **ECS/Fargate:** Always-on cost and patching overhead outweighed the any benefits at the coursework scale.

V. DETAILED SUBSYSTEM DESIGN

A. Sensor simulator

The simulator generates one JSON record per bin at each time step. You can control the interval, number of zones and bins, and even simulate the heat conditions—for example, forcing the bins into a the fire-risk state for a demonstration purposes. The output can be sent directly to the fog process or saved for later use.

B. Fog state and smoothing

For each bin, a deque of length `SMOOTHING_WINDOW` stores the recent raw readings and these smoothed values are used for the feed into scoring. The system then combines the fill level, temperature, and the priority score to determine a single `status` value. The fog layer also tracks the previous alert level, so it only sends an `alert` when the situation escalates, rather than on every update."

C. Validation layer

Validation is intentionally duplicated in this: the same rules underpin Lambdas behaviour and the pytest test cases. Top-level fields in the every payload includes `schema_version` (must equal `1.0`), `message_type`, a numeric `emitted_at` timestamp, and a `records` array.

D. Processor and idempotency (scope note)

Because the system uses at-least-once delivery, the processor may receive the duplicate messages. In a production system, this would require some deduplication or idempotent writes so that it does not happen. This limitation is acknowledged here rather than ignoring it.

E. Read API surface

Table II lists the HTTP routes from the SAM template. Methods and the path parameters follows the standard API Gateway routing conventions.

F. Frontend behaviour

The SPA polls each JSON endpoint on a timer for updating the new data. Navigation routes separate the fleet view in individual bin detail, and the critical lists. Badges for fill and thermal state are styled to be readable at a glance in the UI. There was one recurring integration headache which is worth noting is: because Vite bakes the `VITE_API_BASE_URL` into the bundle at build time, a misconfigured environment variable produces a fully loaded UI that silently fails on every data fetch. This caught me out more than once and is documented for future reference. Figure 2 shows the deployed dashboard with the fleet and critical views is visible.

VI. IMPLEMENTATION

A. Repository layout

Table III maps the each major directory to its role. Cross-cutting models and constants live under `shared/` so that the fog, tests, and backend code all works from the same definitions.

B. AWS deployment practice

Deployment uses the `sam build` followed by the `sam deploy`, with parameters partuclarly configured for the Learner Lab environment (pre-existing deployment role ARN — LabRole, explicit region selection). Stack outputs include the HttpApi base URL, which is needed when configuring the fog scripts and the Amplify environment when deploying frontend.

C. Continuous integration

GitHub Actions runs Python tests, `sam validate`, and `npm run build` on every push. These checks often catches broken templates before they reach remote branches. Amplify builds independently on push to the main branch; keeping Actions green avoids the confusing situation where things work in the Amplify but not locally.

D. Developer experience choices

Used Python 3.12 for the Lambda runtime and it also matches the current AWS guidance. TypeScript strictness on the frontend for type checks and all so it catches undefined fields early in development. Keeping the ingest validation logic in a pure function which makes it easy to test in isolation without deploying anything.

TABLE II
HTTP API ROUTES (REPRESENTATIVE).

Method	Path	Purpose
POST	`/ingest`	Accept fog payloads; validate; enqueue to SQS
GET	`/bins`	Fleet snapshot for the overview screen
GET	`/bins/{bin_id}`	Detail view for one bin
GET	`/zones/{zone}/bins`	Zone-scoped list (uses the GSI)
GET	`/critical`	Bins that need immediate attention
GET	`/history/{bin_id}`	Recent events for charts
POST	`/admin/purge`	Reset demo data (guarded in template)

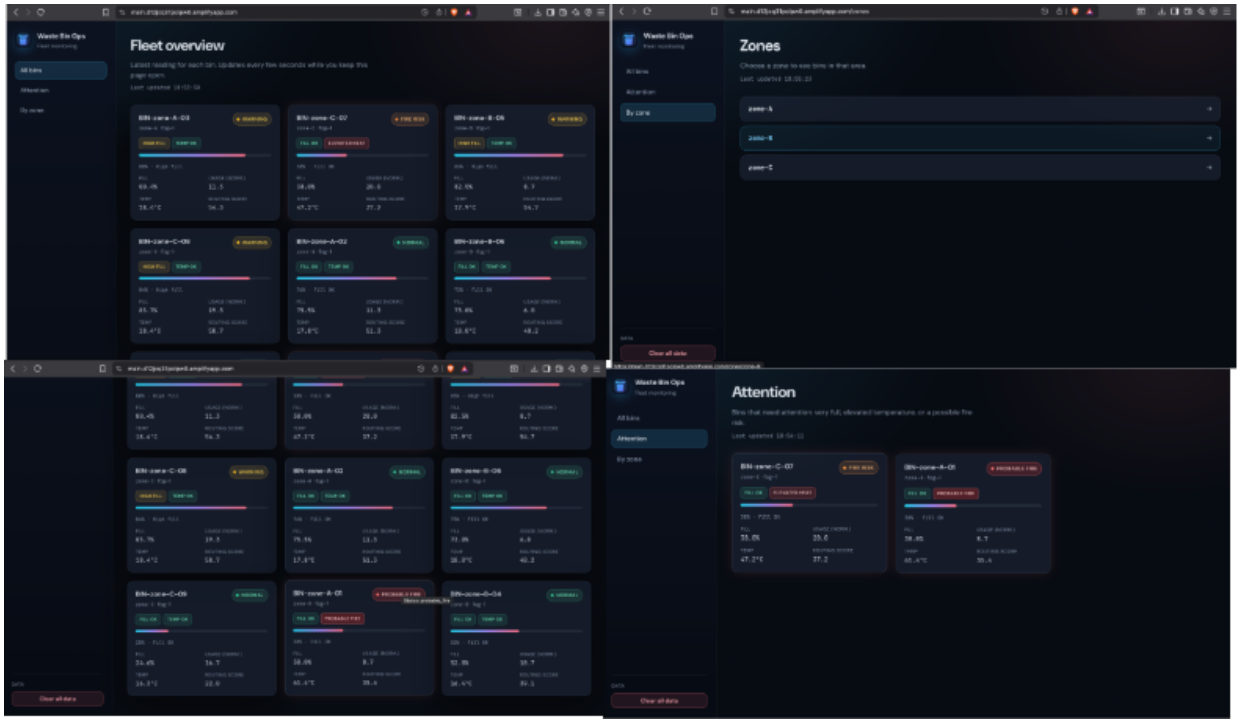


Fig. 2. React dashboard against the deployed read API (fleet/critical views; badges reflect fill and thermal state).

TABLE III
REPOSITORY STRUCTURE (ABBREVIATED).

Path	Contents
`shared/`	Models, constants, shared validation logic
`sensors/`	Simulator CLI and thermal profiles
`fog/`	Fog CLI, smoothing, priority, HTTP client
`backend/`	SAM template, Lambdas (ingest, processor, read)
`frontend/`	Vite + React + TypeScript UI
`tests/`	Pytest coverage for validators and helpers
`scripts/`	Helper shell scripts for demo runs
`amplify.yml`	Amplify build spec (at repo root)

VII. TESTING STRATEGY

A. Unit and contract tests

The pytest suite covers the representative valid and invalid JSON cases: if its missing required keys, wrong `schema_version` values, fill percentages out of range, invalid status enums, and malformed record types. These run in the seconds in CI and give good confidence that the obvious schema drift will be caught well before a demo.

B. Manual integration tests

Integration testing was done manually in a structured sequence: deploy the stack; run the fog agent against `/ingest`; check SQS metrics and Lambda invocation counts in CloudWatch; inspected DynamoDB items directly; open the Amplify URL; compare what the UI shows against `curl` output for the same `bin_id`.

C. Limitations of the testing approach

No automated load testing (e.g. with k6) was performed. No chaos experiments—like throttling DynamoDB or cutting off SQS—were run. The findings here are therefore functional rather than statistical.

VIII. Evaluation

A. Methodology

Evaluation followed the structured checklist: (E1) ingest the happy path working end-to-end; (E2) ingest it correctly rejecting malformed payloads; (E3) processor writing visible rows in both DynamoDB tables; (E4) the read endpoints returning the coherent and consistent JSON; (E5) SPA navigation and statuses matching the API; (E6) UI is recovering correctly after an API URL change and rebuild.

B. Observed failure modes

The most common user-visible failure came from the misconfigured Amplify environment variables which discussed earleir: the site would load fine but every JSON fetch would silently fail until a full rebuild. Expired temporary credentials caused 403 errors sometimes that had nothing to do with the application code itself. Pointing the CLI at the wrong region that led to the "empty" DynamoDB views when the data was actually sitting in a different region or account.

C. Threats to validity

The use of synthetic data means real-world issues like packet loss or clock differences on hardware are not captured. The system's security is only suitable for the demonstration purposes only. Performance claims are based on the design reasoning rather than the measured results.

IX. Conclusions and Future Work

The project delivers a working end-to-end project and pipeline with a fog-shaped preprocessing stage sitting in front of a serverless AWS backend, consistent with the layered fog/cloud model described in the literature [2], [3]. Fog smoothing and the alert escalation was cut down on the redundant data reaching the DynamoDB, and SQS provides the clean decoupling between the synchronous API and the persistence layer. The most durable takeaways is that they were integration-focused: uses queues for decoupling, shares validation logic with the tests, and treat SPA build-time configuration as a first-class concern. Managed scaling simplifies demos considerably, but failure modes like the throttling or the cold starts only really show up under real loads—any future work should include measured load tests before making claims about headroom.

Obvious next steps would include proper MQTT ingress, authenticated reads, WebSocket or server-sent events to replace polling, DynamoDB TTL for ageing out old events, dead-letter queues for handling poison messages, and k6 or Locust experiments to quantify actual latency distributions and per-thousand-ingest costs.

References

References

[1] L. Atzori, A. Iera, and G. Morabito, "The internet of things: A survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.

[2] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the internet of things," in *Proc. Workshop Mobile Cloud Comput. (MCC)*, 2012, pp. 13–16.

[3] A. V. Dastjerdi, H. Gupta, R. N. Calheiros, S. K. Ghosh, and R. Buyya, "Fog computing: Principles, architectures, and applications," in *Internet of Things: Principles and Paradigms*, Elsevier, 2016, pp. 61–75.

[4] OASIS, "MQTT Version 3.1.1," Oct. 2014. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

[5] Amazon Web Services, "AWS Serverless Application Model Developer Guide," 2025. [Online]. Available: https://docs.aws.amazon.com/serverless-application-model/

[6] Amazon Web Services, "API Gateway HTTP APIs," 2025. [Online]. Available: https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api.html

[7] Amazon Web Services, "AWS Lambda Developer Guide," 2025. [Online]. Available: https://docs.aws.amazon.com/lambda/latest/dg/welcome.html

[8] Amazon Web Services, "Amazon SQS Developer Guide," 2025. [Online]. Available: https://docs.aws.amazon.com/sqs/

[9] Amazon Web Services, "Amazon DynamoDB Developer Guide," 2025. [Online]. Available: https://docs.aws.amazon.com/dynamodb/

[10] Meta and contributors, "React," 2025. [Online]. Available: https://react.dev/

[11] E. You and contributors, "Vite," 2025. [Online]. Available: https://vitejs.dev/

[12] H. Gupta, A. V. Dastjerdi, S. K. Ghosh, and R. Buyya, "iFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, Edge and Fog computing environments," *Software: Practice and Experience*, vol. 47, no. 9, pp. 1275–1296, 2017.

[13] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

Appendix A
Illustrative ingest JSON


```
{
  "schema_version": "1.0",
  "message_type": "summary",
  "emitted_at": 1710000000.0,
  "records": [
    {
      "bin_id": "Z1-B01",
      "zone": "Z1",
      "fill_level": 72.5,
      "weight_kg": 41.2,
      "temperature_c": 38.0,
      "usage_rate": 12.0,
      "priority": 54.3,
      "status": "warning",
      "fill_alert": "warning",
      "fire_risk": false,
      "probable_fire_alert": false,
      "timestamp": 1710000000.0
    }
  ]
}
```